Smart Budget Planner

Bachelor of Technology
in
Computer Science & Engineering
School of Engineering &Sciences

Submitted by

Paruchuru Deepthi Sri
AP24110011684

Course Name: Coding Skills

Under the Guidance of
Prakash

SRM University-AP
Neerukonda,Mangalagiri,Guntur
Amaravati,Andhra Pradesh-522240

April 30,2026

Introduction

In today's fast-paced world, making efficient financial decisions is essential, especially when working within a limited budget. Individuals often face challenges in selecting the best combination of items that provide maximum value without exceeding their financial constraints.

The Smart Budget Planner is designed to address this problem using a systematic and algorithmic approach. This project applies the 0/1 Knapsack Algorithm, a fundamental concept in Dynamic Programming, to determine the optimal set of items that maximize overall value within a given budget. By considering each item's cost and importance, the system ensures that users make informed and optimal choices. Unlike manual selection methods, which may lead to inefficient outcomes, this solution guarantees accuracy and efficiency.

Additionally, the project provides multiple interfaces, including command-line, graphical, and web-based versions, making it user-friendly and versatile. Overall, it demonstrates the practical application of theoretical algorithms in solving real-world optimization problems.

Objective

The primary objectives of this project are:

- To design a system that helps users make optimal purchasing decisions
- To implement the 0/1 Knapsack algorithm using Dynamic Programming
- To maximize total value within a fixed budget constraint
- To provide a user-friendly interface (GUI + Web)
- To visualize how dynamic programming works using a DP table
- To demonstrate how theoretical algorithms can solve real-world problems

Problem Statement

The problem addressed in this project is an optimization problem.

Given:

- A set of items, each having:
  - Price (cost)
  - Value (importance score)
- A fixed budget (capacity constraint)

Goal:

Select a subset of items such that:

- Total price ≤ Budget
- Total value is maximum

Nature of Problem:

- This is a combinatorial optimization problem
- Each item can either be:
  - Included (1)
  - Excluded (0)

Methodology

The system follows a structured approach:

Step 1: Input Collection

- User enters the budget
- Items are taken from:
  - Predefined dataset OR
  - User-defined custom inputs

Step 2: DP Table Construction

- A 2D table dp[i][w] is created:
  - i → number of items considered
  - w → budget capacity

Step 3: Decision Making

For each item:

- Check:
  - Skip item
  - Include item (if budget allows)

Step 4: Optimal Value Calculation

- Fill the DP table iteratively
- Choose the maximum value at each step

Step 5: Backtracking

- Traverse the DP table backwards
- Identify which items were selected

Step 6: Output Display

- Selected items
- Total cost
- Remaining budget
- Maximum value

Algorithm Explanation

The project uses the 0/1 Knapsack Algorithm, a classic Dynamic Programming approach.

Core Idea:

At each step, we decide whether to include an item or not based on which choice gives a better result.

Recurrence Relation:

```None
dp[i][w] = max(
   dp[i-1][w],
   dp[i-1][w - weight[i]] + value[i]
)
```

Explanation:

- dp[i-1][w] → Excluding the item
- dp[i-1][w-weight[i]]+value[i] → Including the item

Base Case:

```None
dp[0][w] = 0
dp[i][0] = 0
```

System Design

Modules:

1. knapsack.py

- Implements DP logic
- Handles table creation and backtracking

2. items_data.py

- Stores predefined dataset of items
- Each item includes name, price, value

3. cli_version.py

- Command-line interface
- Displays DP table and results

4. main.py

- Tkinter-based GUI
- Provides interactive user experience
- Allows adding custom items

5. index.html

- Web-based interface
- Visual representation of results and DP table

Technologies Used

Technology	Purpose
Python	Core Programming
Tkinter	GUI development
HTML,CSS,Javascript	Web Interface
Dynamic Programming	Optimization Logic

Features

- User input for budget
- Predefined items dataset
- Add custom items dynamically
- Optimal selection using DP
- DP table visualization
- GUI + Web support
- Real-time results display

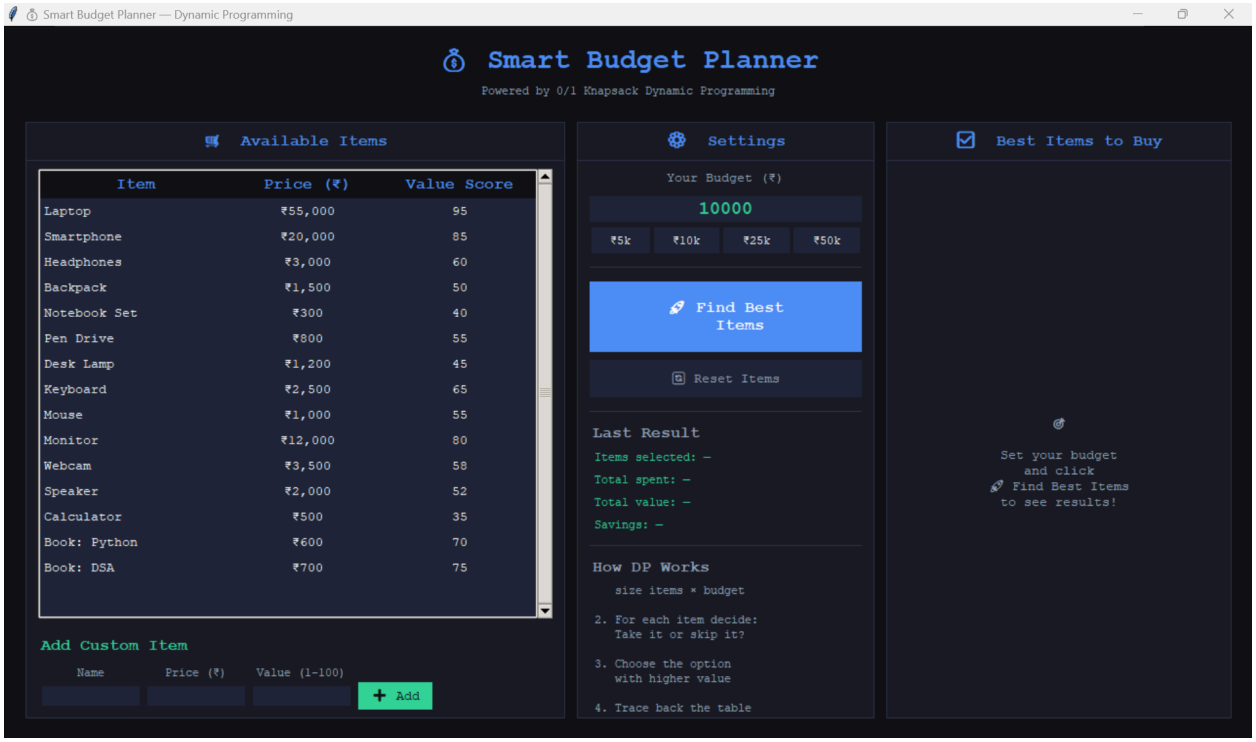
Output Explanation

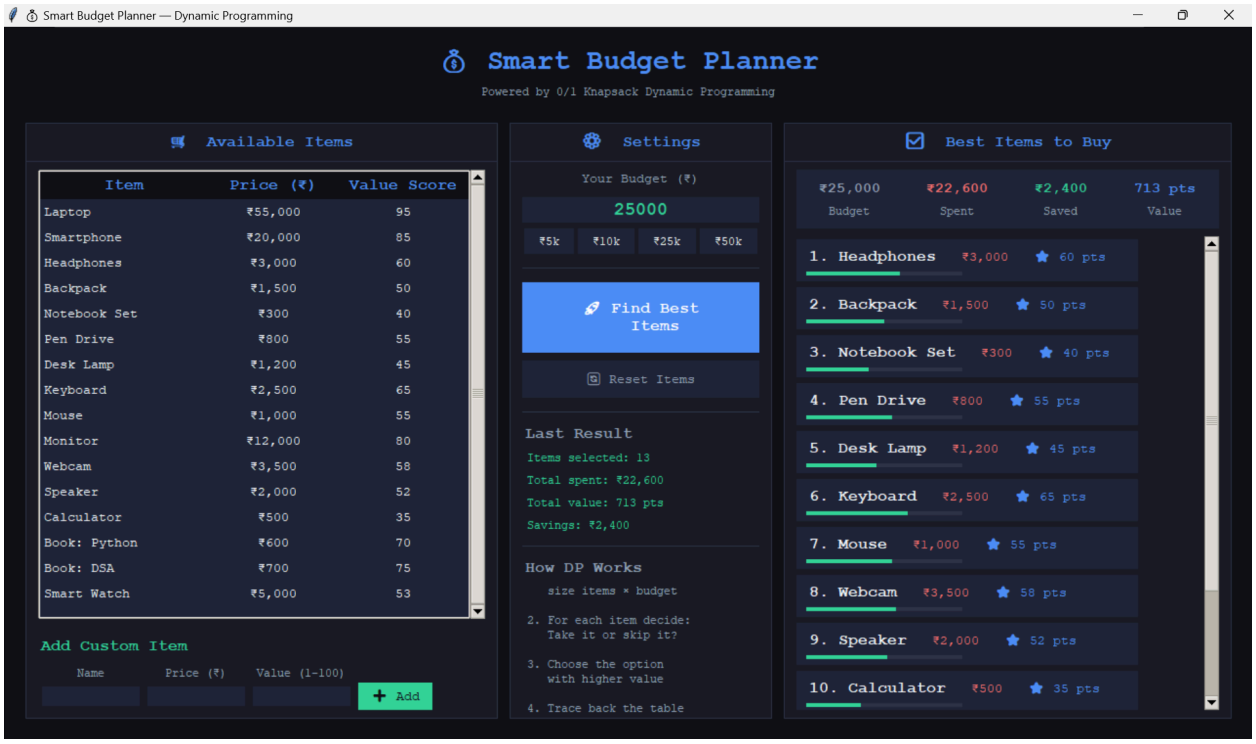
The system outputs:

- Selected Items → Optimal set of items
- Total Spent → Sum of selected item prices
- Remaining Budget → Budget - spent
- Total Value → Maximum achievable value

This ensures the user gets the best combination within constraints.

Output Screenshots





Conclusion

The Smart Budget Planner project successfully implements the 0/1 Knapsack Algorithm to solve a real-world optimization problem of selecting items within a limited budget. Through this system, we demonstrated how Dynamic Programming can efficiently compute the best possible combination of items by avoiding redundant calculations and ensuring optimal results.

The project not only achieved its primary objective of maximizing value under constraints but also provided practical insights into algorithm design and implementation. The inclusion of multiple interfaces, such as GUI, and web versions, enhanced usability and made the system more interactive and accessible.

During development, challenges like handling constraints and optimizing performance were addressed effectively. This project strengthened understanding of core concepts like DP table construction and backtracking.

Overall, it highlights the importance of algorithmic thinking in solving everyday problems and serves as a strong foundation for building more advanced optimization-based systems in the future.